

Цель работы

Исследовать протокол TCP и алгоритм управления очередью RED.

Задание

- Выполнить пример с дисциплиной RED;
- Изменить в модели на узле s1 тип протокола TCP с Reno на NewReno, затем на Vegas. Сравнить и пояснить результаты;
- Внести изменения при отображении окон с графиками (изменить цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).

Выполнение лабораторной работы

Описание моделируемой сети:

- сеть состоит из 6 узлов;
- между всеми узлами установлено дуплексное соединение с различными пропускной способностью и задержкой 10 мс (см. рис. 2.4);
- узел r1 использует очередь с дисциплиной RED для накопления пакетов, максимальный размер которой составляет 25;
- TCP-источники на узлах s1 и s2 подключаются к TCP-приёмнику на узле s3;
- генераторы трафика FTP прикреплены к TCP-агентам.

Разработаем сценарий, реализующий модель согласно описанию в Xgraph график изменения TCP-окна, график изменения длины очереди и средней длины очереди.

```

# создание объекта Simulator

set ns [new Simulator]

# открытие на запись файла out.nam для визуализатора nam
set nf [open out.nam w]

# все результаты моделирования будут записаны в переменную nf
$ns namtrace-all $nf

# открытие на запись файла трассировки out.tr
# для регистрации всех событий
set f [open out.tr w]
# все регистрируемые события будут записаны в переменную f
$ns trace-all $f

# Процедура finish:
proc finish {} {
    global tchan_
    # подключение кода AWK:
    set awkCode {
        {
            if ($1 == "Q" && NF>2) {
                print $2, $3 >> "temp.q";
                set end $2
            }
            else if ($1 == "a" && NF>2)
                print $2, $3 >> "temp.a";
        }
    }
    set f [open temp.queue w]
    puts $f "TitleText: red"
    puts $f "Device: Postscript"
    if { [info exists tchan_] } {
        close $tchan_
    }
    exec rm -f temp.q temp.a
    exec touch temp.a temp.q
    exec awk $awkCode all.q
    puts $f "\"queue
    exec cat temp.q >@ $f
    puts $f "\\n\\\"ave_queue
    exec cat temp.a >@ $f
    close $f
    # Запуск xgraph с графиками окна TCP и очереди:
    exec xgraph -bb -tk -x time -t "TCPRenoCWND" WindowVsTimeReno &
    exec xgraph -bb -tk -x time -y queue temp.queue &
    exit 0
}

```

```

proc plotWindow {tcpSource file} {
    global ns
    set time 0.01
    set now [$ns now]
    set cwnd [$tcpSource set cwnd_]
    puts $file "$now $cwnd"
    $ns at [expr $now+$time] "plotWindow $tcpSource $file"
}

# Узлы сети:
set N 5
for {set i 1} {$i < $N} {incr i} {
    set node_($i) [$ns node]
}
set node_(r1) [$ns node]
set node_(r2) [$ns node]
# Соединения:
$ns duplex-link $node_(s1) $node_(r1) 10Mb 2ms DropTail
$ns duplex-link $node_(s2) $node_(r1) 10Mb 3ms DropTail
$ns duplex-link $node_(r1) $node_(r2) 1.5Mb 20ms RED
$ns queue-limit $node_(r1) $node_(r2) 25
$ns queue-limit $node_(r2) $node_(r1) 25
$ns duplex-link $node_(s3) $node_(r2) 10Mb 4ms DropTail
$ns duplex-link $node_(s4) $node_(r2) 10Mb 5ms DropTail

# Агенты и приложения:
set tcp1 [$ns create-connection TCP/Reno $node_(s1) TCPSink $node_(s3) 0]
$tcp1 set window_ 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]
$tcp2 set window_ 15
set ftp1 [$tcp1 attach-source FTP]
set ftp2 [$tcp2 attach-source FTP]

# Мониторинг размера окна TCP:
set windowVsTime [open WindowVsTimeReno w]
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1];
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout;
# Мониторинг очереди:
set redq [[$ns link $node_(r1) $node_(r2)] queue]
set tchan_ [open all.q w]
$redq trace curq_
$redq trace ave_
$redq attach $tchan_

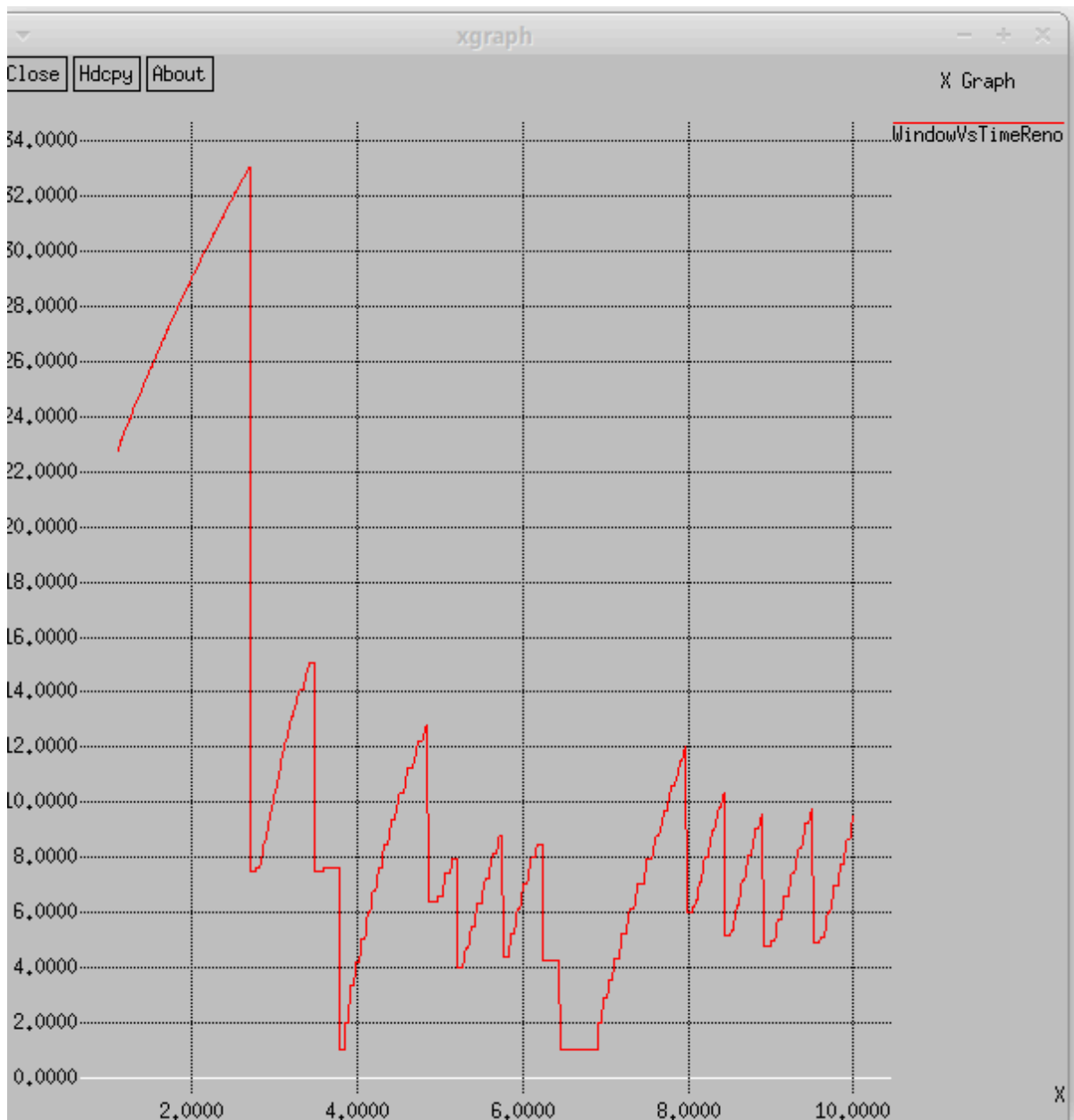
# Добавление at-событий:
$ns at 0.0 "$ftp1 start"
$ns at 1.1 "plotWindow $tcp1 $windowVsTime"
$ns at 3.0 "$ftp2 start"
$ns at 10 "finish"

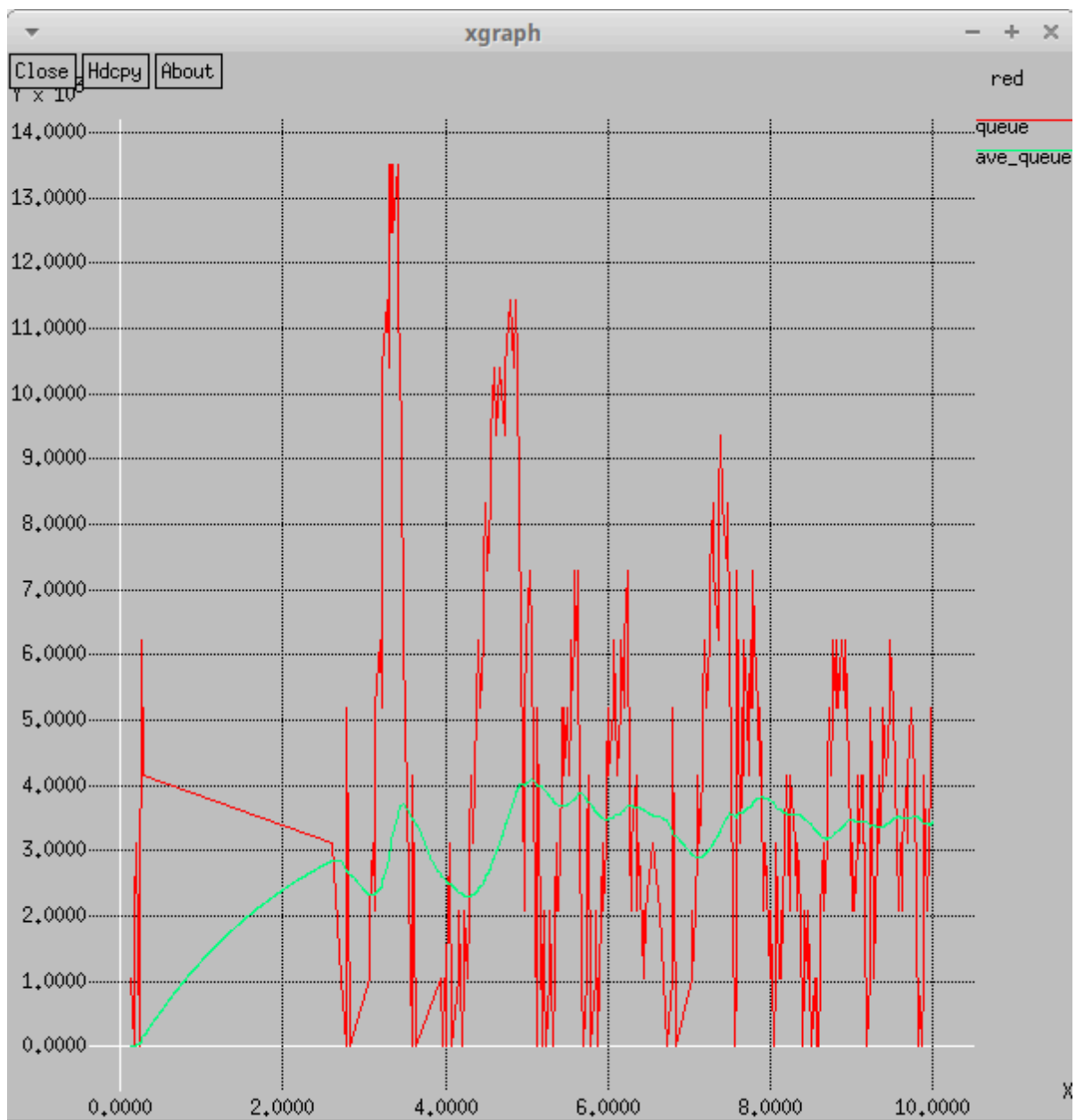
# запуск модели
$ns run

```

После запуска кода получаем график изменения TCP-окна (рис. [-@fig:003]), а также график изменения длины очереди и средней длины очереди (рис. [-@fig:004]).

Здесь видно, что средняя длина очереди находится в диапазоне от 2 до 4. Максимальная длина достигает значения 14.

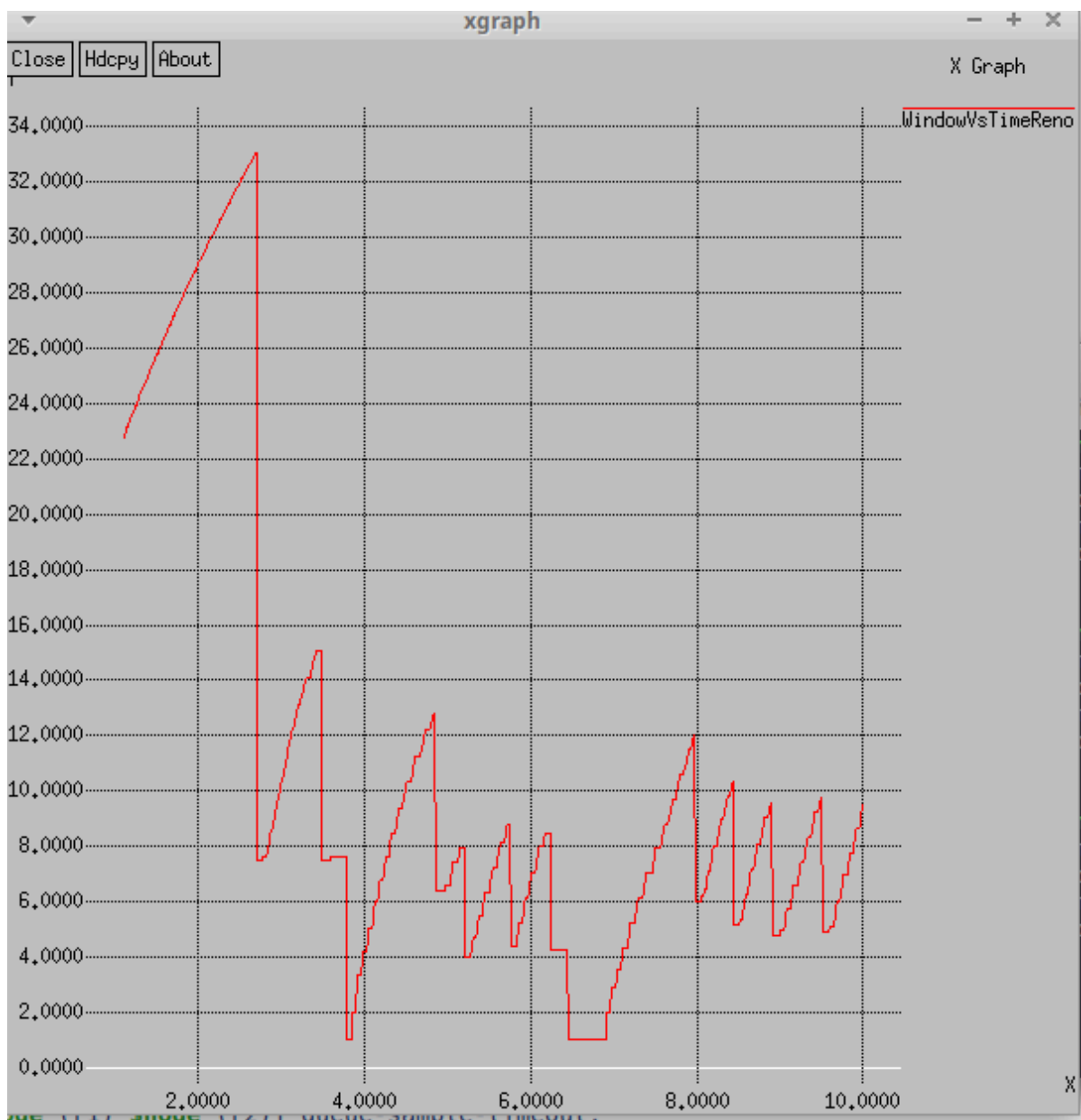


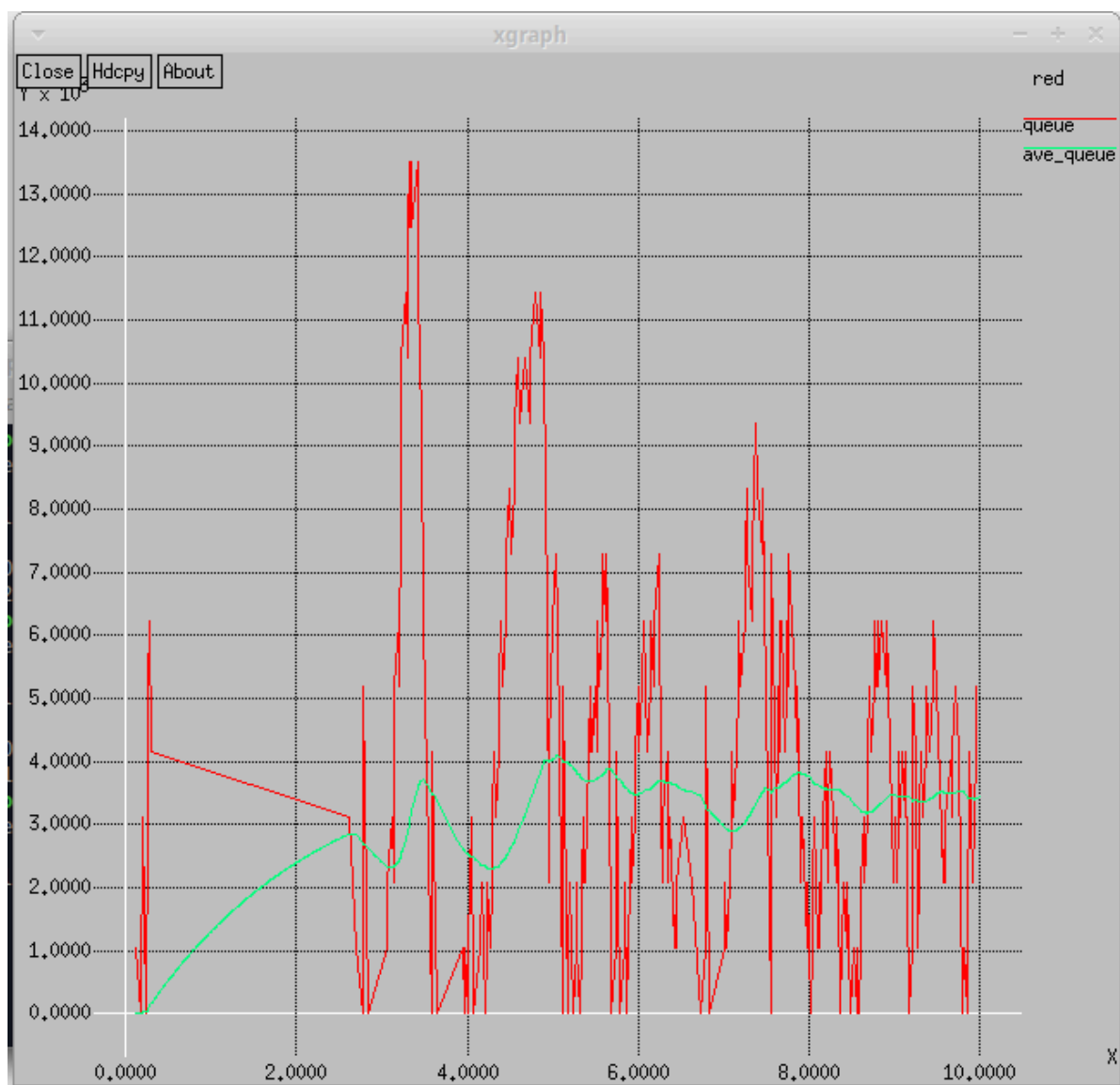


Изменение протокола TCP

Сначала изменила тип Reno на NewReno. В результате получим следующие график изменения TCP-окна (рис. [-@fig:005]), а также график изменения длины очереди и средней длины очереди (рис. [-@fig:006]).

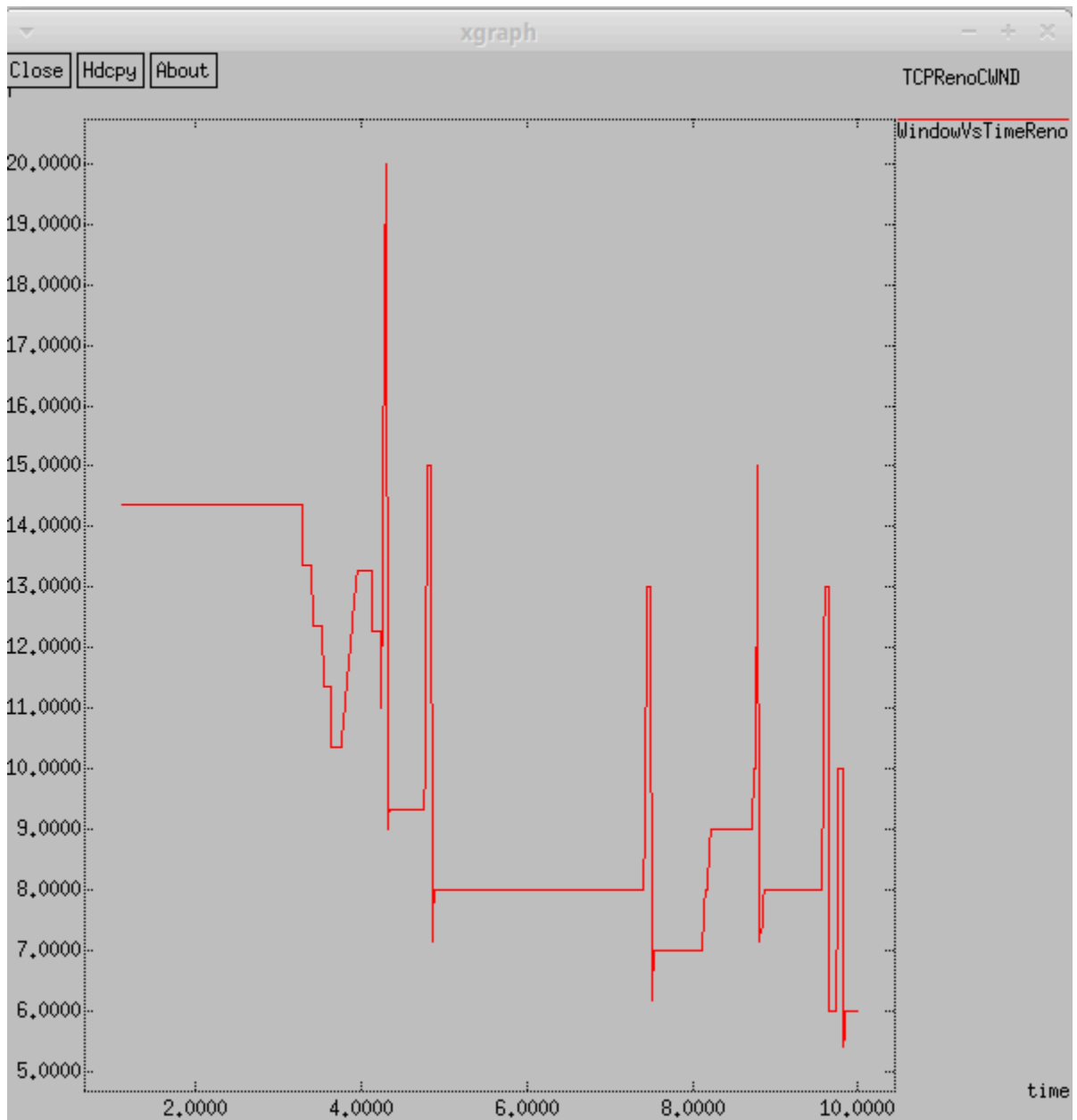
Так же, как было в графике с типом Reno значение средней длины очереди находится в пределах от 2 до 4, а максимальное значение длины равно 14.

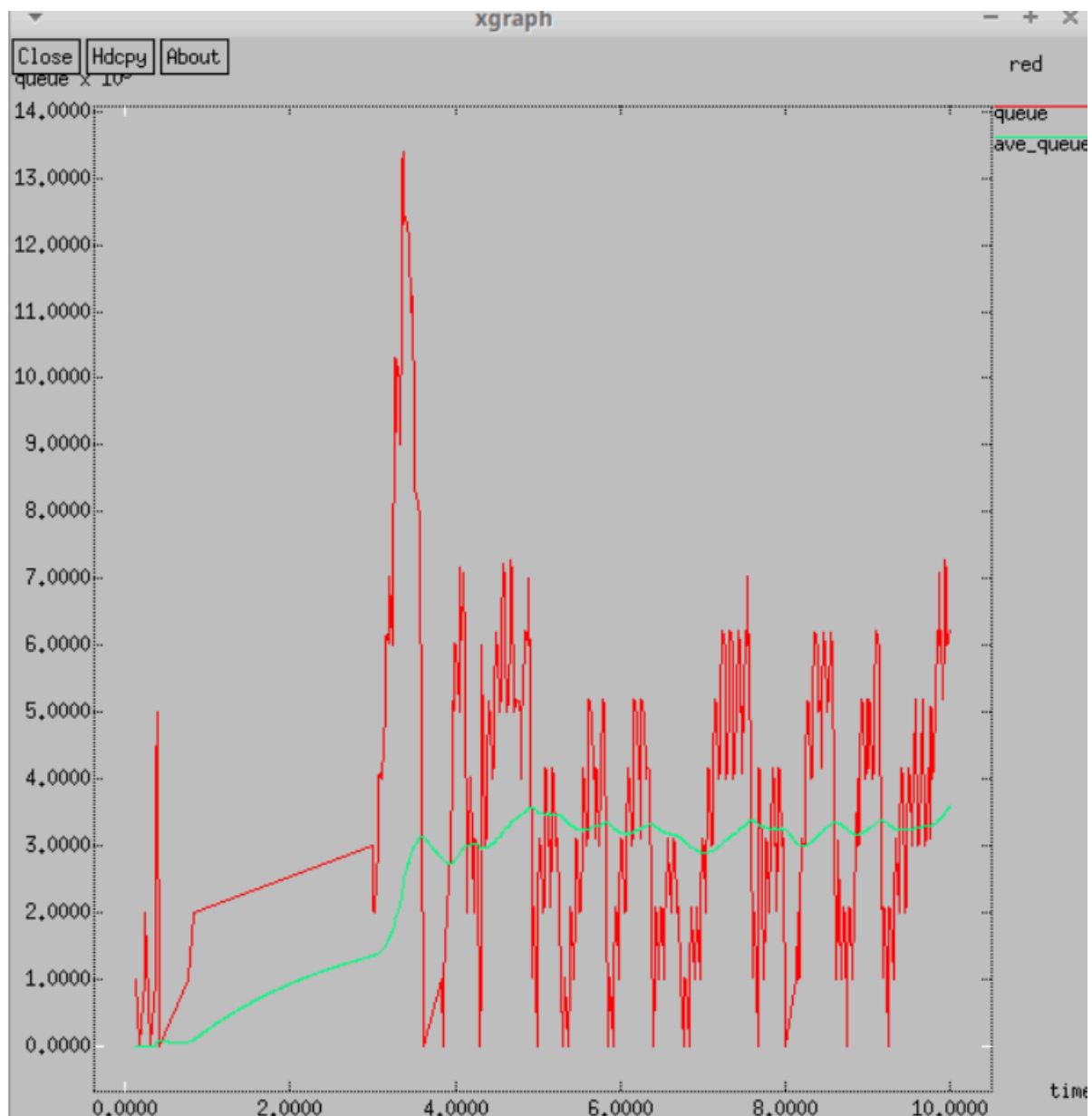




Далее изменим тип Reno на Vegas В результате получим следующие график изменения TCP-окна (рис. [-@fig:007]), а также график изменения длины очереди и средней длины очереди (рис. [-@fig:008]).

Средняя длина очереди при TCP Vegas остаётся в пределах 2–4, максимальная — 14. Размер окна у Vegas не превышает 20 (у NewReno — 34). TCP Vegas раньше обнаруживает перегрузку и уменьшает окно без потерь пакетов.





Изменение отображения окон с графиками

Внесём корректировки в отображение графиков: поменяем цвет фона, цвет линий, подписи осей и название траектории в легенде.

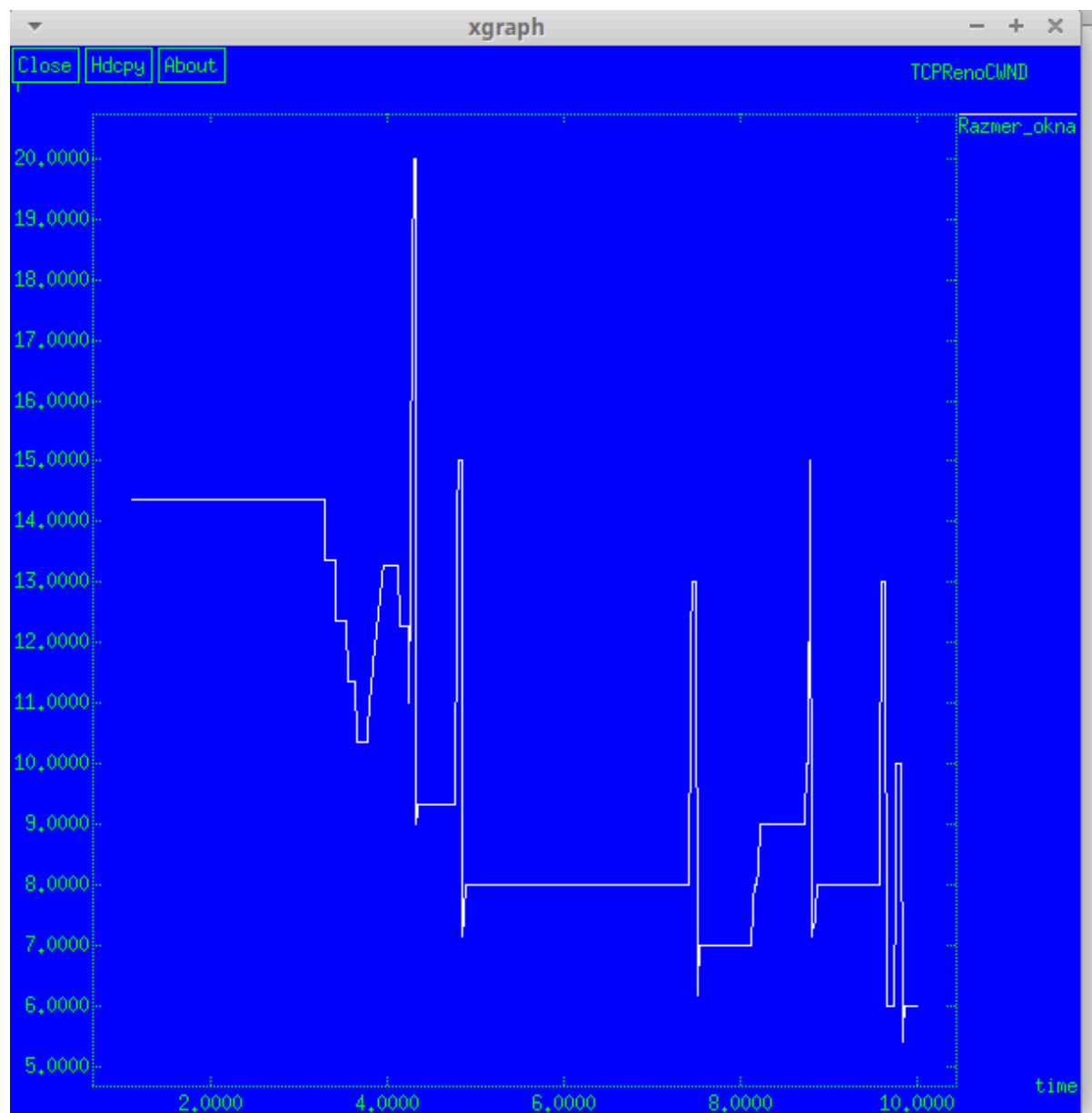
Для этого обновим код: в процедуре `finish` изменим цвет линий и подписей, а также с помощью опций `-fg` и `-bg` зададим новый цвет текста и фона в `xgraph`.

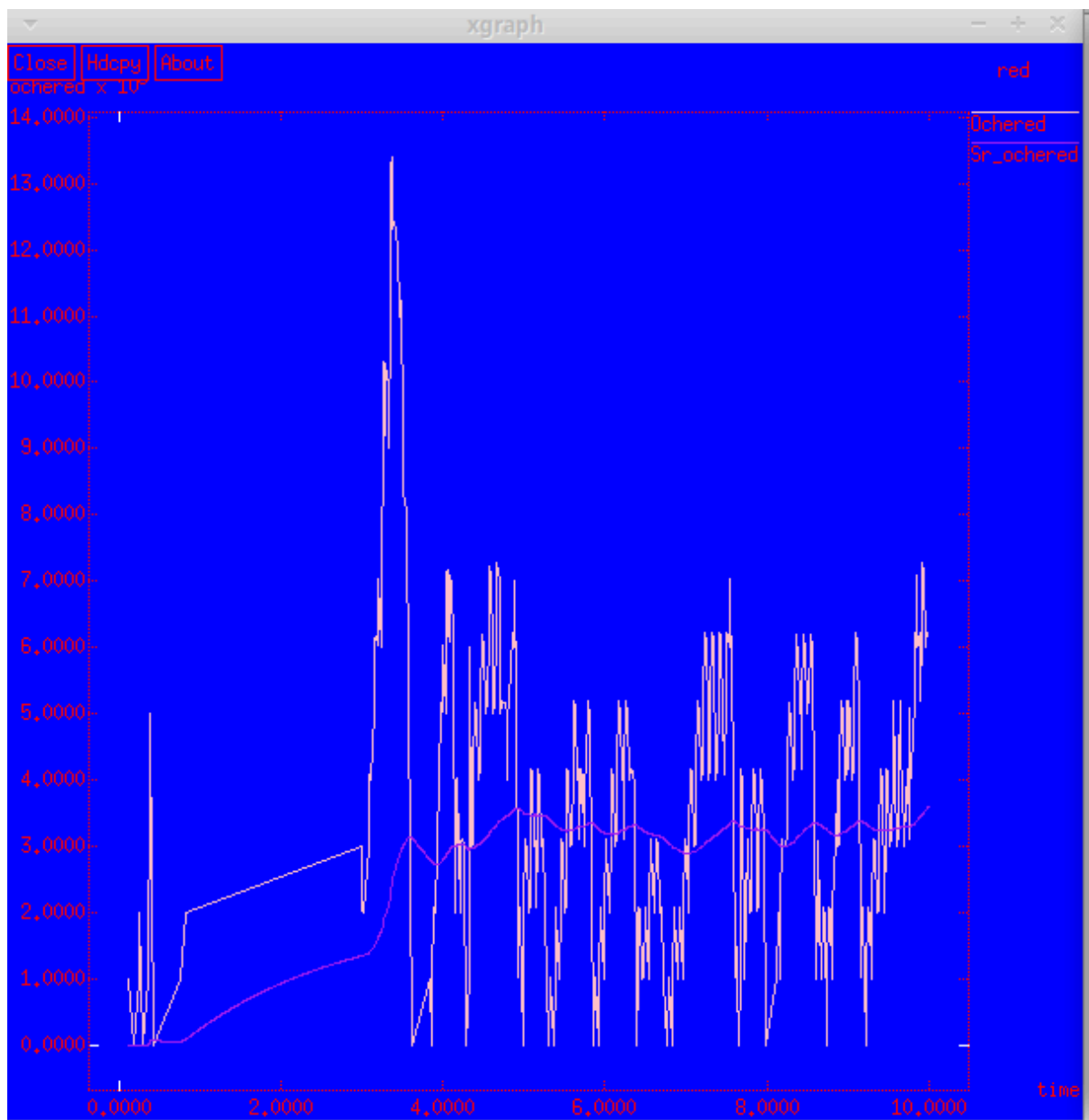
```

}
set f [open temp.queue w]
puts $f "TitleText: red"
puts $f "Device: Postscript"
puts $f "0.Color: Pink"
puts $f "1.Color: Purple"
if { [info exists tchan_] } {
    close $tchan_
}
exec rm -f temp.q temp.a
exec touch temp.a temp.q
exec awk $awkCode all.q
puts $f "\"Ochered\""
exec cat temp.q >@ $f
puts $f "\n\"Sr_ochered\""
exec cat temp.a >@ $f
close $f
# Запуск xgraph с графиками окна TCP и очереди:
exec xgraph -fg green -bg blue -bb -tk -x time -t "TCPReNoCWND" WindowVsTimeReno &
exec xgraph -fg red -bg blue -bb -tk -x time -y ochered temp.queue &
exit 0
}

```

В результате получим следующие график изменения TCP-окна (рис. [-@fig:011]), а также график изменения длины очереди и средней длины очереди (рис. [-@fig:010]).





Выводы

В процессе выполнения данной лабораторной работы я исследовала протокол TCP и алгоритм управления очередью RED.

Список литературы{.unnumbered}

::: {#refs}

:::